

Sink-aware Top-K 融合中的顺序语义门禁

脱敏分享版 | 本文讨论 MoE Router 中 sink expert、Top-K 融合、梯度验证与 canonical ordering，不包含内部模型或部署信息。已完成的多卡验证只是 4 卡 TP2/EP2 算子执行，未进入真实 dispatcher 或短训练。

结论先行

一个 Top-K fusion 即使满足以下条件，也未必能接入真实 MoE dispatcher:

- 选中的 expert 集合相同;
- routing probability 数值一致;
- backward 梯度通过;
- microbenchmark 明显更快。

原因是下游 compact/permute 往往消费**有序索引**，而不只是一个无序集合。某 sink-aware Top-K Triton 原型将 forward launch 从 25 个压到 1 个，在目标 shape 上 forward 提升 3.15 倍、forward+backward 提升 2.09 倍，值与梯度测试 116/116 通过；但 strict `top_indices` slot order 只有 102/116 对齐，canonical order 尚未闭环，因此保持 experimental/default-off。这个 102/116 只表示 Top-K 输出的 slot 顺序检查，不代表已验证完整 compact layout。

这篇文章的重点不是宣传一个已上线优化，而是说明：在路由算子中，“集合等价”与“布局协议等价”是两回事。

1. 为什么 sink expert 会打破现有融合路径

部分 MoE Router 会保留一个需要单独处理的 sink expert。本文只把“sink 的 score 和选择规则与其他 expert 不完全相同”作为算子合同，不推断它的模型设计动机。score 修正、保留规则或概率处理会引入额外分支，使原有融合条件无法直接命中。

未融合路径通常类似：

```
scores
-> sink-specific adjustment
-> group mask / candidate selection
-> Top-K
-> probability gather / normalize
-> compact indices
```

多个 pointwise、mask、Top-K 和 gather 造成 launch 碎片，也会产生中间张量读写。

2. 单 kernel 设计

原型将 sink 语义、候选筛选、Top-K 和概率输出放进一个 Triton kernel，并提供配套 backward：

```
input scores
|
├─ apply sink rule
├─ select candidates
├─ top-k + explicit candidate ordering
├─ emit probabilities
└─ emit expert indices
```

工程接入还包括：

- Router/config 开关；
- 不满足条件时的 dense-dispatch fallback；
- forward 与 fused backward；
- 独立 correctness/performance harness；
- 路径 trace，确认测试确实命中新 kernel。

算子路径还在 4 卡 TP2/EP2 环境中完成执行验证，但这只说明多卡下算子可运行；它不等于真实 dispatcher、完整 compact/permute 链路或短训练已验证。

3. 为什么单 kernel 优于两阶段原型

Top-K 常见的另一种设计是先按 expert group 筛选，再对保留候选做第二阶段 Top-K。它在候选空间很大时可能有优势，但当前 ungrouped 路径中会增加：

- 中间候选写回；
- 第二次 kernel launch；
- 额外同步与元数据；
- 小规模下无法摊薄的固定成本。

对五个 token 规模复验后，两阶段版本均慢于单 kernel，因此停止接入。保留这个负结果很重要：算法复杂度更低不保证实际 GPU 时间更短，尤其在 launch-bound 路径中。

4. 第一层 oracle：值与梯度

Top-K 融合首先需要验证：

- sink 规则是否和参考实现一致；
- expert probability 是否满足容差；
- 被选中和未选中位置的梯度是否正确；
- tie、mask、空组和边界 token 是否覆盖；
- 不同 dtype 下是否出现非有限值。

目标原型的路由语义与梯度用例为 116/116 PASS。这证明数学输出和 autograd 路径在受测算子范围内成立，但仍不足以上线。

5. 第二层 oracle：canonical ordering

5.1 集合相同并不等于张量相同

假设两个实现都选择 expert {2, 5, 7}：

```
reference: [2, 5, 7]
optimized: [5, 2, 7]
```

作为集合它们相同；如果概率与索引同步重排，最终加权和在理想数学上也可能相同。但真实 dispatcher 通常会据此执行：

```
top-k indices
-> flatten token-expert pairs
-> compact / sort / prefix sum
-> build expert-major layout
-> dispatch / combine
```

顺序变化可能改变 compact slot、通信排列、逐位累加顺序、跨 rank 比较和后续缓存。因此 canonical ordering 是接口协议的一部分。

5.2 Strict top_indices slot-order 门禁

在要求 top_indices 的每个 Top-K slot 与 reference 逐项、逐序一致的测试中，只有 102/116 通过。剩余用例不是“数值小误差”，而是 slot 内索引顺序不同。该测试尚未构造并比较完整 compact layout；即使最终 weighted sum 暂时一致，也不能据此证明真实 dispatcher 与训练轨迹不受影响。

6. 性能结果应怎样表述

指标	Reference	Fused	结果
Forward	432.52 μ s	137.10 μ s	3.15×
Forward+backward	487.94 μ s	233.05 μ s	2.09×
Forward GPU launches	25	1	-24
Value/gradient	—	116/116	PASS
Strict top_indices slot order	—	102/116	未通过

这些是算子/代理环境结果，不代表完整训练吞吐。正确结论是“性能潜力明确，但协议门禁未闭环”，而不是“训练提速 2.09×

7. 修复 canonical ordering 的思路

7.1 明确定义 tie-break

Top-K 在相同或近似 score 下必须规定稳定顺序，例如：

```
primary key: score descending
secondary key: expert id ascending
```

参考实现和 Triton 实现要共享同一规则，不能依赖硬件指令或 warp reduction 的隐式顺序。

7.2 保证概率与索引共同重排

任何 canonical sort 都必须成对重排 (probability, expert_id)，否则会产生更隐蔽的概率—专家错配。

7.3 用下游布局做 oracle

除了比较 Top-K 输出，还应直接比较：

- compact slot mapping;
- expert-major token order;
- dispatch counts/prefix offsets;
- combine 后结果;
- 多 rank 下的路径一致性。

8. 上线前的三级门禁

```
Level 1: value + gradient
        116/116 PASS
        |
Level 2: strict top_indices slot order
        102/116 -> STOP
        |
Level 3: real dispatcher + short training
        not entered
```

Level 1/2 包含 4 卡 TP2/EP2 算子执行，但没有进入真实 dispatcher 或短训练。当 Level 2 未通过时，正确动作是停止进入 Level 3，并保持默认关闭。这样可以避免为了追求漂亮的 microbenchmark，将不稳定的隐式协议带入训练系统。

9. 可复用检查表

- ☐ Top-K 的输出契约是集合、稳定序列，还是完整 compact layout?
- ☐ tie-break 是否明确且跨实现一致?
- ☐ probability 与 expert index 是否始终成对重排?
- ☐ 是否覆盖 sink、mask、边界 token、重复 score 和 dtype?
- ☐ backward 是否与参考实现对齐?
- ☐ 是否比较真实 dispatcher 的 slot/offset/order?
- ☐ 是否把算子加速与训练吞吐分开?
- ☐ 未通过严格顺序时是否保持 default-off?

10. 总结

Top-K 是一个接口协议密集型算子。性能优化不仅要复现“选中了哪些 expert”，还要复现下游所依赖的顺序、布局和累加轨迹。这个原型证明单 kernel 能显著减少 launch 并获得可观性能，但 102/116 的 strict `top_indices` slot-order 结果也明确给出了停止条件。目前只完成了 4 卡 TP2/EP2 算子执行；只有 canonical ordering、真实 dispatcher 和短训练全部闭环后，才有资格讨论默认启用。